

RISC-V Out-of-Order CPU - Team MV

ECE 411 mp_000 Final Report

Ansley Tsai

Reuben De Souza

Minsoo Kim

I. INTRODUCTION

Out-of-order (OoO) processors are a cornerstone of modern high-performance computing. Unlike in-order processors, which execute instructions strictly in the sequence they appear in a program, OoO processors dynamically reorder instructions at runtime to maximize hardware utilization and minimize stalls caused by data hazards, cache misses, and other latency sources. This flexibility allows OoO processors to achieve significantly higher instructions per cycle (IPC) compared to their in-order counterparts, making them the dominant design framework in general-purpose CPUs today.

In this project, we designed and implemented a fully functional OoO processor supporting the RV32IM instruction set using an Explicit Register Renaming (ERR) scheme. Starting from a front-end fetch stage and a parameterizable instruction queue, we progressively built out a complete microarchitecture capable of executing instructions out of order while maintaining precise exception semantics through a reorder buffer (ROB). The design was verified against a RISC-V reference model across a suite of benchmarks including Coremark, FFT, AES-SHA, and others.

Beyond the baseline implementation, we developed several advanced features spanning branch prediction, memory system optimization, and pipeline efficiency. These include a GShare branch predictor, a Branch Target Buffer (BTB), a Return Address Stack (RAS), a next-line prefetcher, a pipelined cache, a split load/store queue, and a post-commit store buffer. Together, these optimizations yielded substantial performance improvements over the baseline processor across all six benchmarks, which are analyzed in detail throughout this report.

II. PROJECT OVERVIEW

This project was completed over eight weeks across four biweekly checkpoints, each building on the last to produce a fully functional OoO processor. From the

outset, our primary goal was correctness first and performance second. We wanted a solid, well-verified baseline before investing time in optimization. This philosophy influenced many of our early architectural decisions: we chose ERR over Tomasulo's Algorithm for its cleaner separation of renaming and commit logic, and we opted to implement a split load/store queue from the start rather than building a unified queue and replacing it later, reasoning that the long-term savings in development time outweighed the added upfront complexity.

For project management, we maintained a shared GitHub repository and divided work based on the requirements of each checkpoint, collaborating to integrate individual contributions at each milestone. We pursued advanced features where the performance benefit was clear and the implementation cost was manageable. The features we ultimately implemented span branch prediction, cache hierarchy improvements, and memory system optimizations, and are described in detail in Section III-C.

III. DESIGN DESCRIPTION

A. Overview

Our processor is built around an ERR scheme. ERR maintains a physical register file larger than the 32 architectural registers defined by the ISA. A Register Alias Table maps each architectural register to its current physical register at dispatch time, with a new physical register allocated for each instruction that writes a result. This eliminates WAW and WAR hazards by construction, without requiring values to be written through the ROB. The ROB tracks in-flight instructions and ensures results are committed to architectural state in program order, preserving precise exception semantics. The following subsections describe how this architecture was built up incrementally across four checkpoints.

B. Milestones

1) *Checkpoint 1:* The first checkpoint established the front-end of our processor, with the primary goal of building a correct and modular fetch pipeline to serve as

Thanks to our mentor Jason and rest of ECE411 course staff.

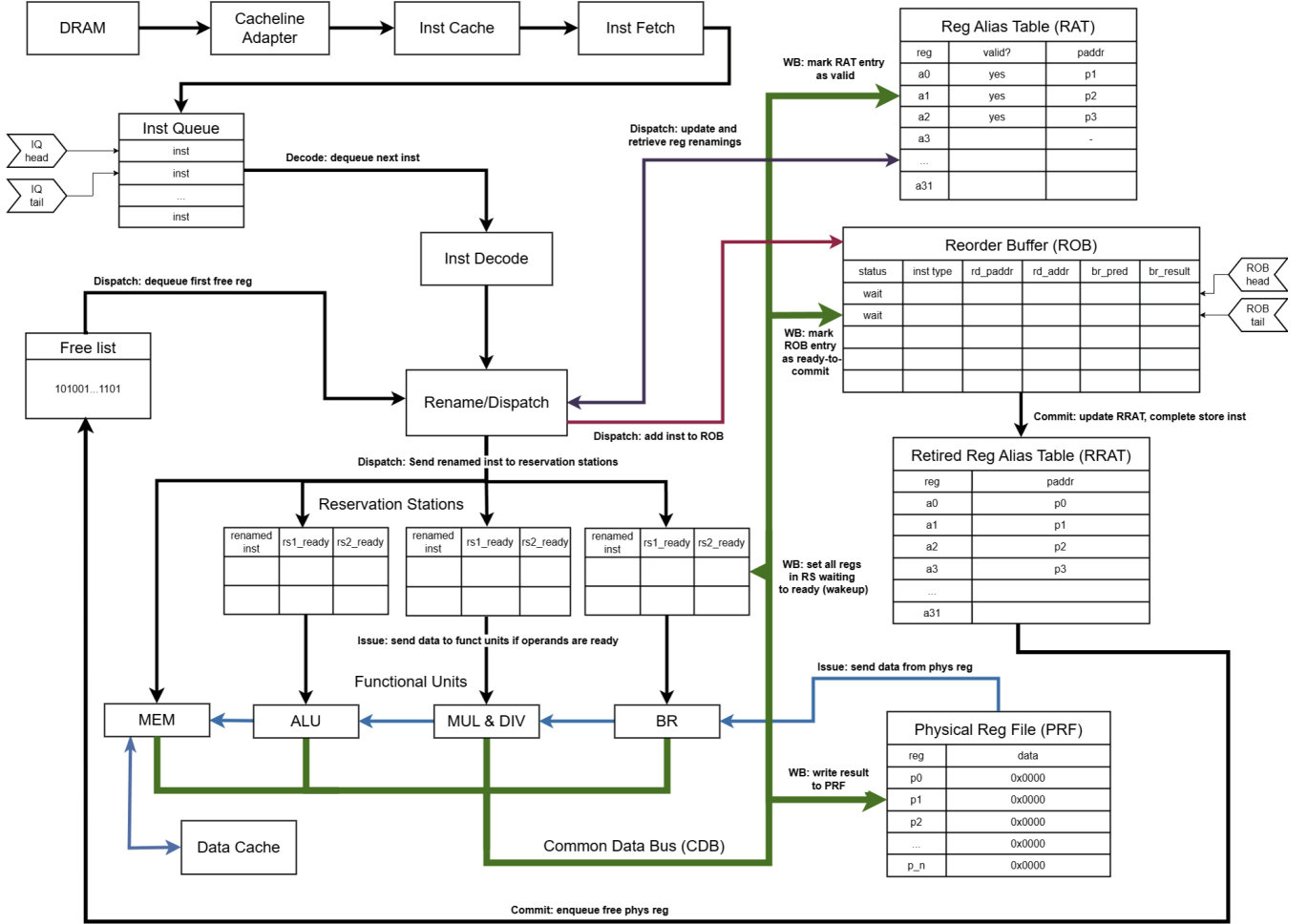


Fig. 1: High-level block diagram for our OoO processor. Note that MUL and DIV are actually separated in our final design; they are combined here to simplify the diagram.

a stable foundation for the execution engine developed in later checkpoints. A high-level diagram of the full processor is shown in Fig. 1.

The fetch stage retrieves instructions sequentially from the program counter, initialized to $0 \times \text{AAAAA}000$. To avoid stalling on every fetch, we implemented a 256-bit linebuffer that caches the most recently fetched cache line, allowing sequential instructions within the same line to be served without issuing a new cache request and achieving 1 IPC for sequential fetches. Fetched instructions are then placed into a parameterizable FIFO queue, from which the decode stage pops one instruction per cycle. Parameterizing the queue by both depth and width made it reusable across different pipeline configurations later in the project.

The cache line adapter interfaces between the cache and the banked burst DRAM model, deserializing burst

responses into full 256-bit cache lines. We chose to issue only one outstanding DRAM request at a time to keep the design simple at this stage. To plan for when the instruction and data caches access DRAM simultaneously, a fixed-priority arbiter favors the data cache, since memory instructions are typically more critical to pipeline forward progress than instruction fetches, which can be absorbed by the linebuffer. Components were verified through waveform-based simulation.

2) *Checkpoint 2:* The second checkpoint implemented the core out-of-order execution engine, including the register renaming infrastructure, reservation stations, functional units, and the reorder buffer. Memory and branch instructions were deferred to Checkpoint 3.

Register renaming is handled by two table-based modules: the Register Alias Table (RAT) and the Retirement Register Alias Table (RRAT). The RAT maps archi-

tectural registers to physical registers at dispatch time, while the RRAT tracks committed architectural state and is used to restore the RAT on a flush. Physical registers are managed using a bitmask-based freelist, where each bit represents the availability of one physical register. We chose a bitmask over a FIFO-style freelist because recovery on a flush is simpler. The freelist can be reconstructed directly from the RRAT rather than rolling back a queue.

The ROB was built on top of the parameterizable queue from Checkpoint 1, extended with index-specific write logic to allow individual entries to be updated as instructions complete out of order. We implemented separate reservation stations for ALU, multiply, and divide operations, all instantiated from the same parameterizable module. Each reservation station uses a struct tailored to its functional unit, avoiding unnecessary control signals, and the separation ensures that long-latency operations like division cannot block dispatch of ALU or multiply instructions. For execution, we integrated Synopsys DesignWare sequential multiplier and divider IPs configured for 3 and 16 cycle latencies respectively. Since reservation station entries are freed as soon as an instruction is dispatched to its functional unit, in-flight metadata for these multi-cycle operations is tracked using a set of registers that update based on the start and complete signals provided by the IPs.

Rather than implementing a shared common data bus (CDB) with arbitration logic, each functional unit contributes to a combined CDB packet broadcast to all structures simultaneously, with an enable bit indicating whether its contribution is valid. This avoided arbitration complexity at the cost of some additional area, which was acceptable at this stage.

Correctness was verified using two complementary approaches: waveform analysis of the provided `ooo_test.s` to confirm instructions were completing out of order, and a Python script that generated randomized assembly programs over the supported instruction subset, which were run against the RVFI/Spike monitor to systematically check correctness across a wide range of instruction sequences.

3) *Checkpoint 3*: The third checkpoint completed the baseline processor by adding support for memory and control flow instructions, with correctness as the primary objective.

Rather than implementing a unified load-store queue, we directly implemented a split LSQ from the start, as we had planned to add this as an advanced feature and saw no benefit in building a simpler version first.

In the split LSQ, store operations are dispatched into a parameterizable FIFO queue and begin execution once the store at the head of the queue has reached the head of the ROB. The store commits at the ROB once the data cache acknowledges the write. Load instructions are dispatched into their own reservation station and issued once their operands are ready and all preceding stores have committed, at which point the target address is computed and a read request is issued to the data cache.

Branch instructions are handled similarly to ALU instructions, with a dedicated reservation station and functional unit that computes the branch outcome and target address. If a misprediction is detected, the corresponding ROB entry is flagged, but the flush is deferred until the mispredicted branch reaches the head of the ROB, at which point a `flush_pipeline` signal is broadcast to all stateful structures. For this checkpoint, we implemented a static not-taken prediction, with dynamic branch prediction added later as an advanced feature.

Recovery is performed by copying the RRAT back to the RAT and reconstructing the freelist bitmask from the physical registers tracked in the RRAT. One edge case arises when the mispredicted branch itself has a destination register writeback, like with `JAL`, which writes `pc+4` to a destination register. To handle this correctly, the writeback data is forwarded directly into the RAT restoration logic, ensuring the restored RAT reflects the correct post-commit state including the branch’s own writeback. On a flush, the caches are also notified to discard any in-flight responses; since the cache address cannot be changed mid-cycle, the flush causes the cache to drop the current response and reissue on the corrected PC the following cycle.

Correctness was verified against the six provided benchmarks (Coremark, AES-SHA, Compression, FFT, Mergesort, and Image) using the RVFI/Spike monitor to check every instruction commit.

C. Advanced Features

The final phase of the project focused on implementing optimizations across at least three feature categories, with the goal of beating the course baseline on both IPC and PD^4 across all six benchmarks. Table I summarizes the features we implemented and their associated point values.

Performance was evaluated using the six benchmarks provided by the course staff. IPC and delay were measured through simulation, while critical path timing and power consumption were obtained through synthesis.

TABLE I: Advanced Features

Advanced Feature	Category	Points
Split LSQ	Memory Unit	4
Pipelined Cache	Cache Hierarchy	4
Next Line Prefetcher	Prefetching	2
GShare Branch Predictor	Branch Prediction	4
Branch Target Buffer	Branch Prediction	2
Return Address Stack	Misc.	2
Pre-commit Store Buffer	Memory Unit	2
Misaligned Store Forwarding	Memory Unit	1
Post-Commit Store Buffer	Cache Hierarchy	5
Total		26

To streamline this process, we wrote a Tmux script to run all six benchmarks in parallel, with results tracked in a shared spreadsheet for comparison across design iterations.

1) *Split Load Store Queue*: As described in Section III-B3, we implemented a split load/store queue directly at the Checkpoint 3 stage rather than starting with a unified queue. The split LSQ separates loads and stores into independent structures: stores execute in order while loads can issue as soon as their operands are ready and all preceding stores have committed, allowing loads to execute out of order relative to one another as shown in Fig. 2.

Because we did not implement a unified LSQ baseline, we cannot report a direct before/after IPC comparison for this feature in isolation. However, memory access patterns from benchmark profiling suggested that store commits and cache writeback latency were significant sources of pipeline stalls, which motivated other the desire for other features like a pipeline cache, prefetching, and post-commit store buffer.

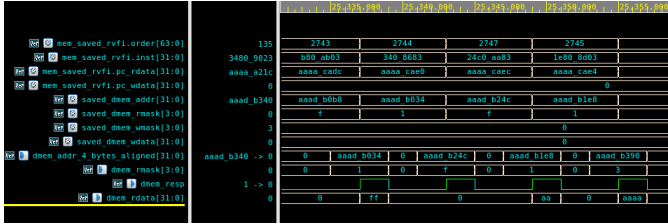


Fig. 2: Waveform showing out of order loads. Because the rs1 operands are ready earlier, a load instruction with order 2747 is executed before a load instruction with order 2745.

2) *Pipelined Cache*: We pipelined our data cache to sustain one access per cycle on consecutive cache read hits. The primary implementation challenge was ensuring that the address and mask of the next memory

instruction finish computing in the same cycle that the cache responds to the previous request. To resolve this, we added a pending slot that holds the next memory instruction while the cache is busy, so that its address can be computed in parallel. Once the cache responds, the pending request is promoted to an in-flight request in the same cycle. A secondary challenge was handling read-after-write hazards; we addressed this by stalling the pipeline for one cycle after a write to allow the data to finish committing to the cache before the next read proceeds. Fig. 3 shows an instance of 1 access per cycle throughput in practice.

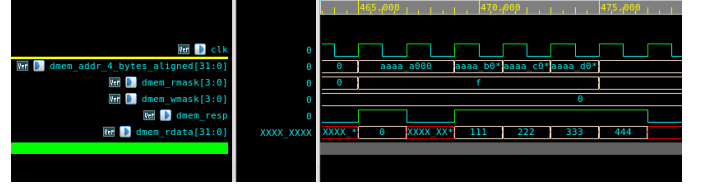


Fig. 3: Waveform 1 access per cycle throughput in pipelined cache. A new request is being sent in the same cycle the cache has responded to the previous request.

In isolation, the pipelined cache produced modest IPC improvements across most benchmarks, as shown in Fig. 4. This is likely because consecutive cache hits on distinct memory instructions are relatively uncommon in our benchmark suite. The benefit is only realized when a new memory request is ready to issue in the same cycle the cache responds, which requires both high cache hit rates and a ready instruction in the pending slot. However, the pipelined cache provided greater gains when combined with other features, particularly the post-commit store buffer, where the increased store throughput created more opportunities for back-to-back cache accesses.

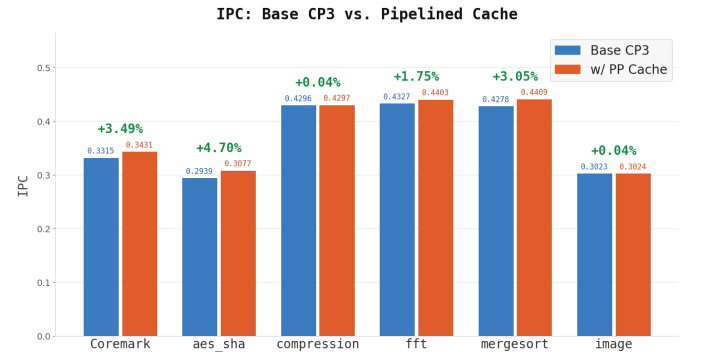


Fig. 4: IPC metrics between our baseline CP3 and pipelined cache.

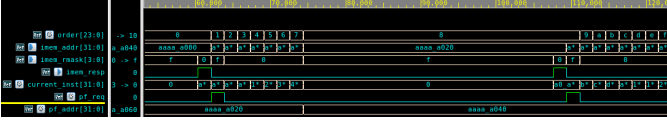


Fig. 5: Waveform demonstrating next line instruction prefetching. Note how instruction read-mask is asserted high once the cache is free, allowing the next cache line to be prefetched while the processor is busy executing the 8 instructions in the current cache line.

3) *Next Line Prefetcher*: We implemented a next-line prefetcher for our instruction cache, motivated by the observation that instructions are fetched sequentially in the absence of branches. When the instruction cache is not servicing an outstanding request, the prefetcher speculatively issues a request for the next sequential cache line before it is explicitly demanded, as shown in Fig. 5.

To implement the prefetcher, we extended the line buffer from a single cache line to a two-slot structure, where slot 0 holds the most recently demanded cache line and slot 1 holds a speculatively prefetched line. A prefetch request is issued when no demand miss is currently in flight, the target line is not already present in either slot, and no prefetch for that line is already in flight. The line buffer tracks the in-flight prefetch address to suppress duplicate requests until the response arrives, and distinguishes between demand and prefetch fills on the response to update the correct slot.

Across our standard benchmarks, the prefetcher produced modest IPC improvements in isolation, as shown in Fig. 6. The limited gains are unsurprising given the branch-heavy nature of our workloads. A taken branch invalidates the prefetched line entirely, since the speculatively fetched sequential address is no longer on the correct execution path. To quantify the upper bound of prefetcher benefit, we tested against a 50,000-instruction sequence with no branches, where the improvement over baseline was substantial. As with the pipelined cache, we expect greater combined gains when the prefetcher operates alongside branch prediction, which reduces the frequency of prefetch invalidations.

4) *Branch Prediction*: Rather than applying a single predictor to all control flow instructions, we implemented a class-based prediction scheme where each type of control flow instruction in RV32IM is handled by a dedicated mechanism. The goal was to achieve meaningful prediction accuracy across all branch types rather than optimizing heavily for one at the expense of others.

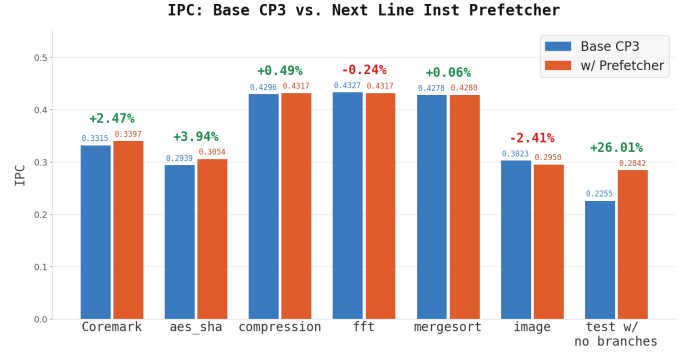


Fig. 6: IPC metrics between our baseline CP3 and instruction prefetcher.

The following sections describe each predictor and its performance characteristics.

To avoid adding a cycle to the fetch stage, all prediction structures are read speculatively in parallel with the instruction cache, indexed by `pc_next`. This means every fetched instruction triggers a read of the GShare PHT and BTB regardless of whether it is actually a branch, at the cost of increased power from unnecessary SRAM reads. The RAS is read combinatorially and incurs no additional latency. Concurrently, the predecoder examines the opcode bits of the fetched instruction to classify it as a conditional branch, JAL, or JALR, and computes statically known target addresses by combining the decoded immediate with the current PC. The instruction class then determines which prediction output is used to redirect the PC: the GShare direction for conditional branches, the RAS top entry for function returns, or the BTB target for indirect JALR instructions. A predicted taken branch immediately redirects the PC, invalidating the linebuffer and signaling the prefetcher to abort its current request since the prefetched line is no longer on the correct execution path.

GShare: Conditional Branch Direction. Conditional branches are handled by a GShare predictor, which correlates branch history with the branch PC to index into a Pattern History Table (PHT). Our implementation uses a 7-bit Global History Register (GHR) and a PHT with 128 two-bit saturating counter entries, indexed by $PC[8:2] \oplus GHR$. We found that higher-order PC folding onto the GHR did not yield meaningful accuracy improvements for our benchmark suite, so we opted for the simpler lower-bit indexing scheme.

Both the GHR and PHT are updated on commit rather than speculatively. While this means the GHR may lag behind true branch history during speculative execution,

it eliminates the need for GHR snapshots and recovery logic on a misprediction flush. This was a complexity trade-off we found acceptable given the modest accuracy cost.

We selected a PHT size of 128 entries based on initial benchmarking, as it provided a favorable accuracy-to-power ratio given that the PHT is implemented in SRAM. As shown in Fig. 7, accuracy improves modestly with larger PHT sizes across most benchmarks. In retrospect, the PD^4 analysis in Fig. 8 shows that 256 entries would have been the optimal choice, offering lower PD^4 than 128 entries while avoiding the significant power overhead incurred at 512 and 1024 entries. This sweep was performed after the final design was submitted due to time constraints.

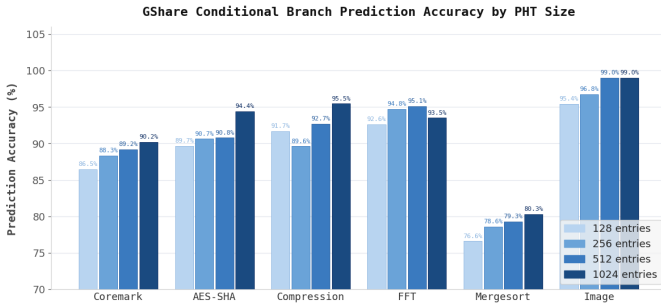


Fig. 7: Conditional branch prediction accuracy vs. PHT size.

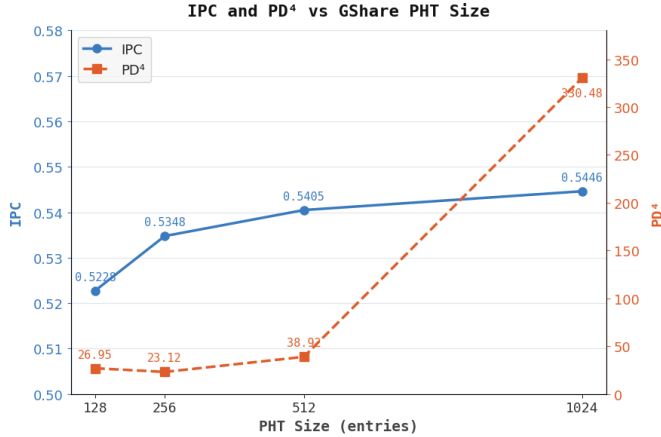


Fig. 8: IPC and PD^4 vs. GShare PHT size.

Return Address Stack: Function Returns. Function returns are handled by a Return Address Stack (RAS). We detect function calls by checking for JAL or JALR instructions where the destination register is $x1$, pushing the return address onto the RAS at fetch time. Returns are detected by checking for JALR instructions where

$rd = x0$ and $rs1 = x1$. We verified through profiling that the compiler does not use $x5$ as a link register in our benchmark suite and therefore did not implement the $x5$ calling convention.

RAS recovery on a misprediction flush is handled by storing a copy of the current RAS stack pointer in each instruction’s ROB entry. When a flush occurs, the stack pointer is restored from the flushing instruction’s ROB entry, ensuring the RAS remains consistent with committed architectural state.

The RAS achieves an average accuracy of 93.64% across most benchmarks, with the notable exception of Compression, which contains only 2 function calls and returns and therefore sees little prediction benefit. Increasing the RAS depth from 16 to 32 entries yields only a 0.05% average accuracy improvement while incurring a 6mW power penalty from the additional flip-flops, so we retained the 16-entry design as the more power-efficient choice.

Branch Target Buffer: Indirect Jumps. The remaining class of hard-to-predict control flow is indirect JALR instructions that are neither function returns nor have statically determinable targets. These are handled by a Branch Target Buffer (BTB) implemented in SRAM with 32 entries, each storing 30 bits of target address (the lower 2 bits are omitted as they are always zero for valid instruction addresses). A separate flip-flop valid array is maintained alongside the SRAM to avoid reading uninitialized entries before the BTB has been populated. During fetch, JALR instructions are first checked against the return detection logic and if identified as a return, the RAS is used; otherwise the BTB is consulted for a valid predicted target and updated on commit with the resolved address.

As shown in Table II, the BTB achieves high accuracy on most benchmarks but performs poorly on AES-SHA and Compression. Compression contains only 3 non-return JALR instructions, all of which suffer compulsory cold-start misses, resulting in effectively zero useful predictions. AES-SHA, on the other hand, makes heavy use of function calls across many distinct call sites, causing significant thrashing in the 32-entry BTB as entries evict one another. Expanding the BTB to 256 entries improved JALR prediction accuracy on AES-SHA to 79% (5193 of 6574 indirect jumps), suggesting the 32-entry BTB is undersized for this workload. However, the area and power costs of a 256-entry SRAM were not justified given the modest overall impact on benchmark performance, so we retained the 32-entry design.

Overall Performance. Branch prediction was imple-

TABLE II: JALR BTB prediction accuracy (32 entries).

Benchmark	Accuracy
Coremark	99.44%
AES-SHA	15.45%
Compression	0%
FFT	99.85%
Mergesort	99.83%
Image	91.21%

mented and evaluated on top of a core that already included the pipelined cache and next-line prefetcher, so the numbers reported here reflect the combined effect of these features. Table III summarizes per-class prediction accuracy across all benchmarks, and Fig. 9 shows the cumulative IPC improvement as each predictor is added.

GShare provides the largest single IPC uplift, as conditional branches are the most frequent control flow instruction across all benchmarks. The RAS and BTB contribute additional gains on benchmarks with frequent function calls and indirect jumps respectively, though their impact is more workload-dependent. Notably, AES-SHA sees a slight IPC regression when adding the BTB due to thrashing, as discussed above. Despite this, the combined power overhead of all three prediction structures is only a few milliwatts, while the reduction in execution time is substantial (average of 17%), delivering overall PD^4 improvements.

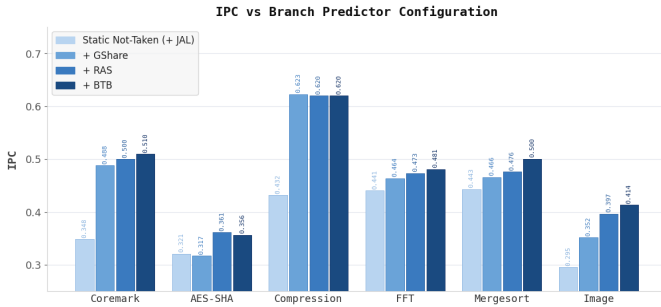


Fig. 9: Cumulative IPC improvement as each branch predictor is added.

5) *Post Commit Store Buffer*: In our Checkpoint 3 baseline, a store must wait for the data cache to acknowledge the write before it can commit at the ROB. While this avoids hazards, it significantly limits throughput during write misses, as the ROB head stalls until the cache responds. The post-commit store buffer addresses this by decoupling store commits from cache writes: once a store’s operands are ready, it is enqueued into the store buffer and allowed to commit at the ROB immediately, while the store buffer retains the meta-

data and drains to the cache independently. This allows subsequent instructions to advance through the ROB without waiting for cache acknowledgment, improving throughput in write-heavy workloads.

The store buffer drains its head entry to the cache when two conditions are met: the entry’s instruction order is older than the current ROB head (meaning it has already committed), and the cache is not currently servicing a load. Loads are given priority over store buffer drains to avoid stalling the in-flight instruction stream. Since stores have already committed architecturally at enqueue time, cache responses to store buffer drains do not produce a CDB broadcast. Fig. 10 shows four store instructions committing in consecutive cycles before the cache has responded to the first store request, demonstrating the decoupling between ROB commit and cache writeback.

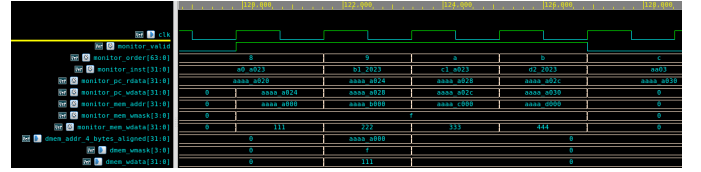


Fig. 10: Waveform showing four store instructions committing in consecutive cycles before the data cache has responded to the first store request, reducing stall pressure on the ROB.

The post-commit store buffer was evaluated in combination with the pipelined cache and next-line prefetcher, as all three features were implemented together by this stage. As shown in Fig. 11, the feature produces significant IPC improvements in write-heavy benchmarks such as FFT and Mergesort, where store commits frequently stalled the ROB in the baseline. AES-SHA sees a slight IPC decrease, consistent with its lower memory write intensity observed during profiling.

6) *Pre-Commit Store Buffer and Store-to-Load Forwarding*: A pre-commit store buffer (SB) holds stores that have finished executing, meaning their target address and data are known but have not yet reached the head of the ROB and are therefore not yet permitted to write to the data cache. Beyond acting as a staging area, the SB enables store-to-load forwarding: when a load issues, it broadcasts its address to all valid SB entries and checks for overlapping stores. If a match is found, the data is forwarded directly to the load’s destination register without requiring a cache access, saving many cycles and allowing dependent instructions to resolve earlier.

Predictor	Coremark	AES-SHA	Comp.	FFT	Merge.	Image
GShare (conditional branches)	85.45%	89.68%	91.69%	92.64%	75.62%	95.44%
RAS (function returns)	99.68%	93.43%	50.00%	93.99%	82.82%	98.28%
JALR BTB (indirect jumps)	99.44%	15.45%	20.00%	99.85%	99.83%	91.21%

TABLE III: Branch prediction accuracy by predictor and benchmark. Each row reports accuracy only over the class of instructions handled by that predictor.

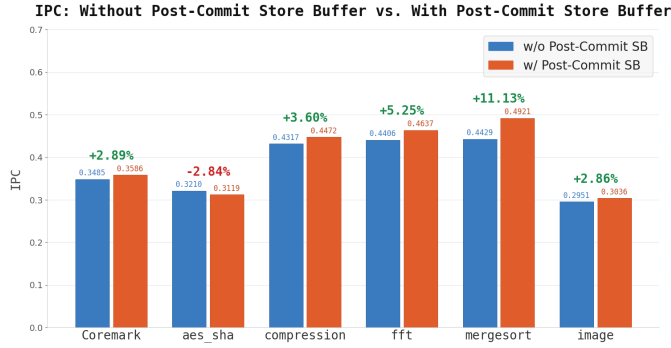


Fig. 11: IPC comparison without and with the post-commit store buffer.

The forwarding logic operates at byte granularity. When a load enters the execute stage, it computes its 4-byte aligned address and compares it against each valid SB entry. A per-byte hit mask is built by scanning entries from oldest to youngest, with newer store data overwriting older data for the same bytes. Three forwarding outcomes are possible: a full hit, where the SB satisfies the entire load mask and the result is forwarded directly to the CDB; a partial hit, where only some bytes are available in the SB and the remainder must be fetched from cache and merged with the forwarded bytes; and a miss, where no relevant SB entry exists and the load proceeds to cache normally.

In the waveforms below, the key signals to observe are `SB_hit` and `SB_partial_hit`, which indicate the forwarding outcome; `cdb_out`, which shows when a result is broadcast to the pipeline; and `dmem_*`, which shows whether a cache access was required. Fig. 12 shows a full hit where a single store satisfies the entire load mask and the result reaches `cdb_out` without a cache access. Fig. 13 shows a partial hit where `SB_partial_hit` is asserted and the cache is still accessed to supply the missing bytes. Fig. 14 shows how newer SB entries correctly overwrite older forwarding data for overlapping bytes. Fig. 15 demonstrates a case where two store halfwords together fully cover a load word's read mask, allowing the entire result to be assembled from SB entries alone and forwarded directly

to `cdb_out` without a cache access.

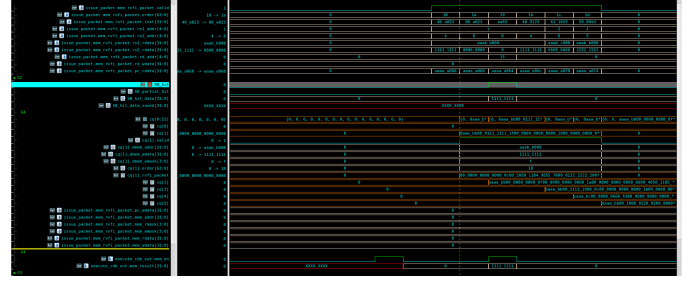


Fig. 12: Full SB hit: `SB_hit` is asserted and the load result is forwarded entirely from the SB to `cdb_out` without a cache access.



Fig. 13: Partial SB hit: `SB_partial_hit` is asserted, the available bytes are forwarded from the SB, and the remainder is fetched from cache and merged.

Despite correct implementation, the feature was ultimately excluded from our final design for several reasons. The IPC improvement across all benchmarks was minimal, as shown in Fig. 16, because the total number of SB hits across our workloads was low. Stores frequently drained to the cache and exited the SB before a dependent load could issue, limiting forwarding opportunities. The feature also introduced compatibility issues with our pipelined cache and caused frequency degradation, making the cost-benefit trade-off unfavorable. A potential improvement would have been to delay

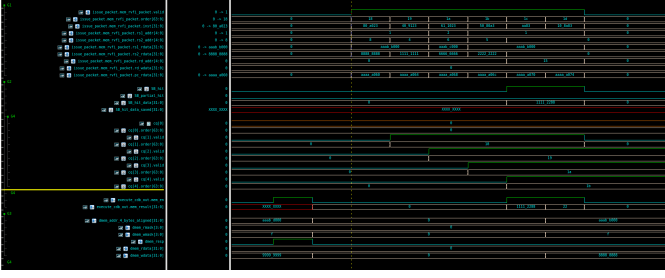


Fig. 14: Newer SB entries overwrite older forwarding data for overlapping bytes, ensuring loads always receive the most recent store value.

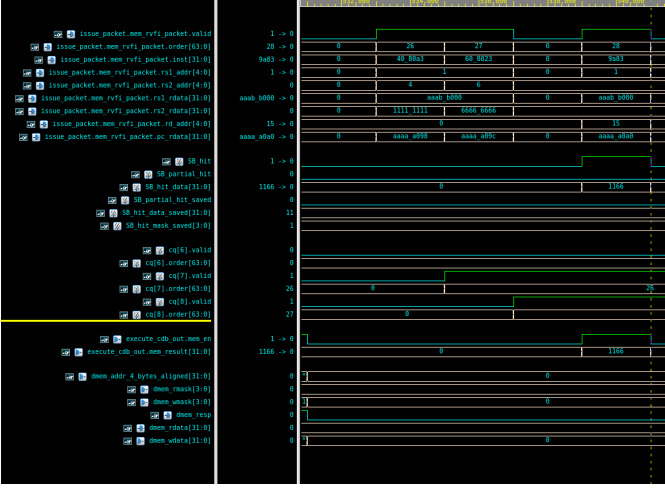


Fig. 15: Two store halfwords whose bytes together fully cover a load word's read mask.

draining the SB to cache until a minimum number of entries were occupied, increasing the window of opportunity for forwarding.

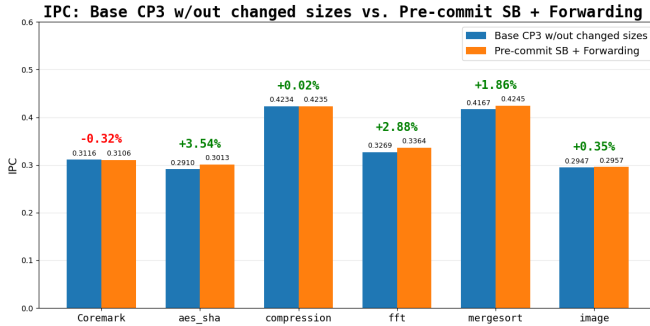


Fig. 16: IPC comparison between the Checkpoint 3 base-line and the pre-commit store buffer with forwarding.

IV. ADDITIONAL OBSERVATIONS

A. Clock Gating

The OpenRAM SRAM model used in this design does not account for the chip select signal (`csb`) when reporting power, meaning SRAMs are modeled as consuming full power regardless of whether they are actually being accessed. To address this, we implemented manual clock gating on the data cache SRAM arrays using glitch-free Integrated Clock Gating (ICG) latches.

The gating logic computes an active-high enable signal for each SRAM array, asserted only when a read or write is actually required — on a new unstalled request, when priming the cache pipeline, or when a dirty miss requires fetching the victim line for writeback. To prevent glitches from propagating to the SRAM clock input, the enable signal is latched on the negative edge of the clock. The gated clock is formed by ANDing the latched enable with the clock, and the inverted latched enable is passed to the SRAM's active-low `csb` port, ensuring the chip select and clock signals remain consistent and avoiding spurious reads or writes. As an expected consequence of ICG latch insertion, this introduces a time-borrowing path flagged by Design Compiler as part of the critical path, though this does not affect functional correctness.

We attempted to apply the same technique to the instruction cache, GShare PHT, and BTB SRAMs, but these efforts were unsuccessful. In some cases the additional logic interfered with the tools' clock tree synthesis optimizations in ways that increased power; in others, the gating introduced glitches that caused programs to hang. Given that these issues arose close to the submission deadline, we left those SRAMs ungated. The data cache clock gating alone yielded a power reduction of approximately 6 mW.

B. Dual Commit

Profiling revealed significant ROB backpressure across our benchmarks: FFT alone experienced over 500K ROB-full stall events. To address this, we implemented dual commit, which allows the top two ROB entries to be retired in the same cycle when both are ready and neither is a branch instruction. Branches are excluded to avoid complicating flush and recovery logic. The change is intentionally conservative and did not impact our critical path, but produces a modest and consistent IPC improvement across all benchmarks as shown in Fig. 17.

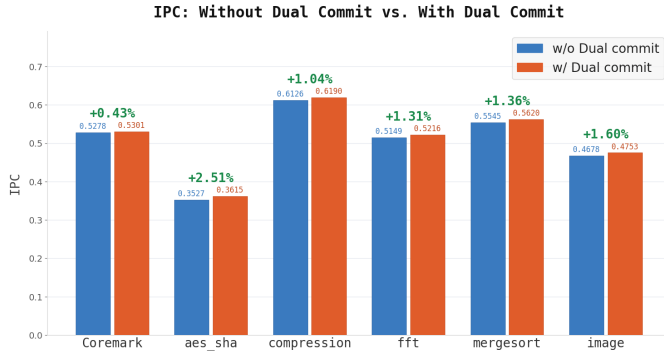


Fig. 17: IPC comparison without and with dual commit.

C. Benchmark Profiling and Optimization

Profiling the instruction mix of our benchmarks was one of the most valuable tools for guiding microarchitectural decisions, though we did not begin doing this systematically until late in the advanced features stage. The insights below reflect what we learned, along with what we would have done differently had we profiled earlier using SPIKE.

Table IV summarizes the instruction mix across all benchmarks. ALU instructions dominate in every workload, followed by memory instructions, with control flow making up the remainder. Multiply instructions appear in only three benchmarks and division in only two, both at low counts relative to ALU operations. This raises several observations about our initial design choices: dedicating a full reservation station and CDB slot to division was likely unnecessary, and those resources could have been better allocated to additional ALU capacity. Similarly, our branch ALU was largely underutilized relative to the integer ALU; a hybrid functional unit handling both would have improved structural utilization without significant added complexity.

TABLE IV: Instruction mix by benchmark (thousands).

Type	Core.	AES	Comp.	FFT	Merge.	Img.
ALU	163K	448K	281K	815K	265K	309K
Memory	71K	299K	89K	254K	130K	35K
Control	60K	26K	66K	33K	98K	86K
Multiply	10K	—	—	65K	—	24K
Division	—	4K	—	—	—	23K

Stall analysis revealed significant structural pressure in FFT, which was the last benchmark we needed to beat. Hundreds of thousands of store queue full, ALU RS full, and load RS full stall events pointed directly to the bottlenecked structures, allowing us to target size increases precisely. After increasing the relevant

structure sizes and adding the post-commit store buffer, these stall events dropped dramatically. This kind of targeted analysis would have been far more effective earlier in development rather than relying on intuition when initially sizing structures.

One optimization that profiling suggested would have meaningful impact, but which we were unable to implement, is issuing loads out of order with respect to stores targeting different addresses. Our split LSQ uses a combinational check to determine memory instruction ordering that created timing pressure and resisted restructuring. A more distributed approach, such as an independence matrix that tracks load-store ordering incrementally, would have spread the logic more evenly across the pipeline and enabled out-of-order load issue more naturally, saving many cycles currently wasted waiting for preceding stores to commit before a non-conflicting load can proceed.

V. CONCLUSION

This project set out to design and implement a fully functional out-of-order processor supporting the RV32IM instruction set using an Explicit Register Renaming scheme, built from the ground up over eight weeks across four checkpoints. Fig. 18 summarizes the performance of our final design against the course baseline. Our design achieves higher IPC and lower PD^4 on every benchmark, with IPC being 4% to 31% higher and PD^4 values being 24% to 69% lower. The gains are the result of the full optimization stack — pipelined cache, next-line prefetching, class-based branch prediction, split load/store queue, and post-commit store buffer — working together rather than any single dominant feature.

In retrospect, earlier and more systematic benchmark profiling would have significantly improved our design process, particularly in sizing microarchitectural structures and identifying the value of out-of-order load execution earlier. A superscalar dispatch and execution pipeline likely remains the most impactful optimization left on the table, and would have required fundamental changes to the fetch, rename, and commit stages that were likely too ambitious given our late consideration. Overall, we are satisfied with both the technical depth of the design we produced and the engineering judgment we developed in the process of building and optimizing it.

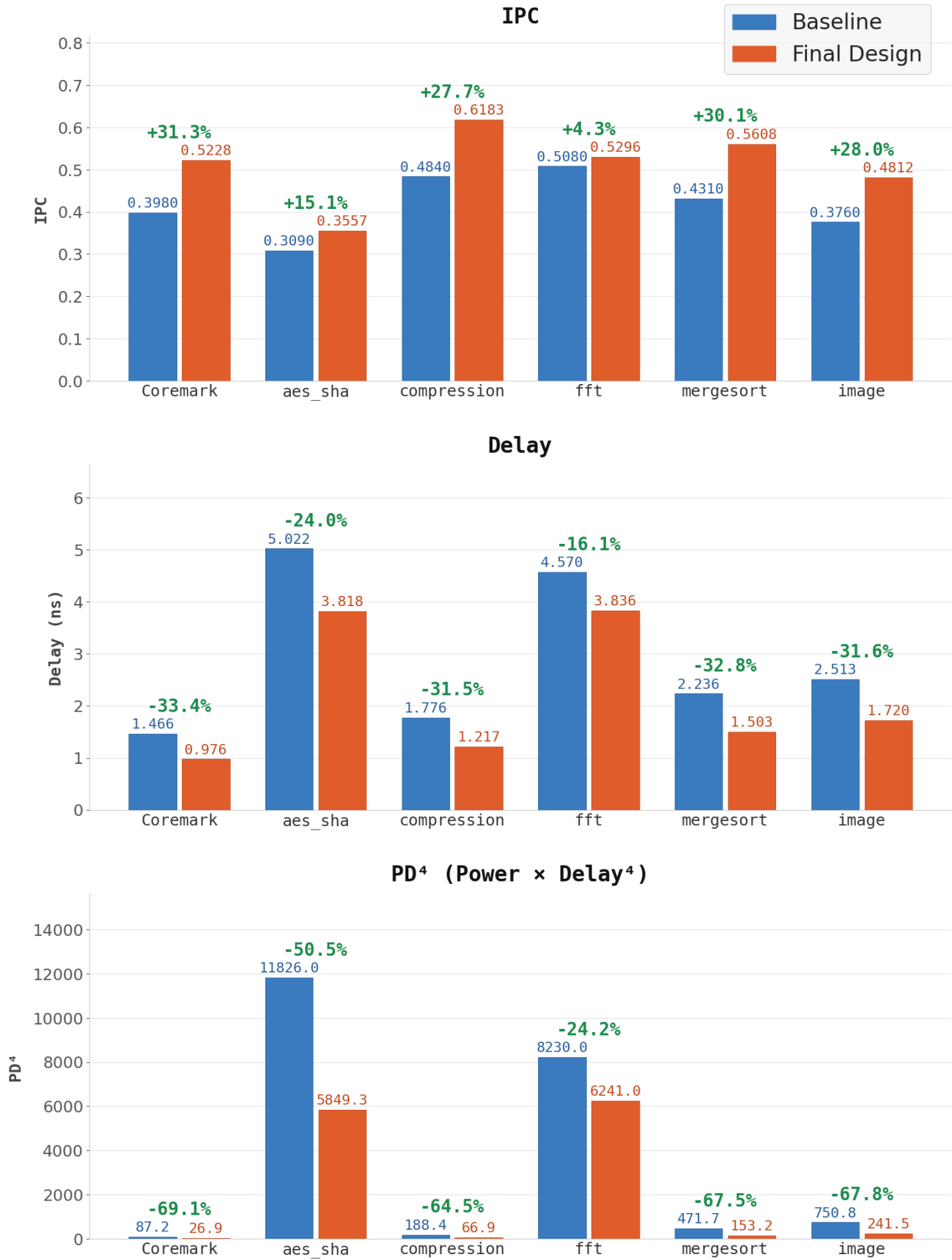


Fig. 18: Final benchmark comparison between the course baseline and our design across IPC, delay, and PD^4 .